

**Hindi Vidya Prachar Samiti's
RAMNIRANJAN JHUNJHUNWALA COLLEGE OF
ARTS, SCIENCE & COMMERCE
(EMPOWERED AUTONOMUS)**

Generative Artificial Intelligence



Name: Janavi P. Vishwakarma

Roll No.: 10440

Class: MSc Data Science and Artificial Intelligence Part-II



Ramniranjan Jhunjhunwala College of Arts, Science and Commerce

Department of Data Science and Artificial Intelligence

CERTIFICATE

This is to certify **Miss. Janavi P. Vishwakarma** of MSc. Data Science and Artificial Intelligence, roll no. **10440** has successfully completed the practical of **Generative Artificial Intelligence** during the Academic Year 2024-2025.

Date: 04-09-2024

Prof. Mujtaba Shaikh
(Prof-in-charge)

External Examiner

INDEX			
Practical No	Practical Name	Date	Signature
1.	Implementation of perceptron	20/06/2024	
2.	Implementation of Classification using ANN: a) Binary b) Multiclass	25/06/2024	
3.	Implementation of Regression using ANN	25/06/2024	
4.	Implementation of Regularization by Early Stopping	05/07/2024	
5.	Implementation of Dropout: a) Classification b) Regression	08/07/2024	
6.	Implementation of RNN	22/07/2024	
7.	Implementation of CNN	13/08/2024	
8.	a) Vanilla Autoencoders b) Denoising Autoencoders	20/08/2024	
9.	Generative Adversarial Network (GAN)	03/09/2024	

PRACTICAL NO.: 01

AIM: Implementation of Perceptron

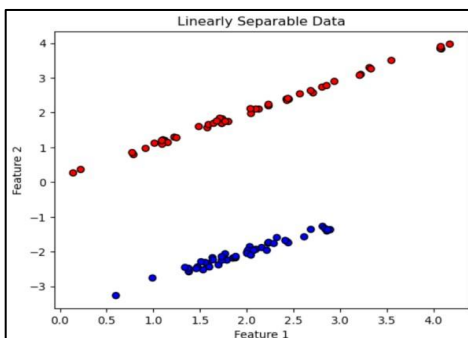
```
import numpy as np import matplotlib.pyplot as plt
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score import
pandas as pd import seaborn as sns import
matplotlib.pyplot as plt
from sklearn.datasets import make_classification
```

Generate linearly separable data

```
X, y = make_classification(n_samples=100, n_features=2, n_informative=2, n_redundant=0,
n_clusters_per_class=1, class_sep=2, random_state=42)
```

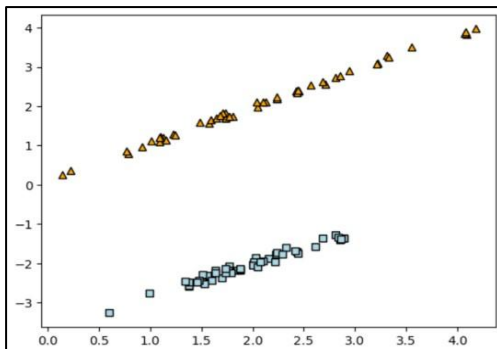
Plot the data

```
plt.scatter(X[:, 0], X[:, 1], c=y, cmap='bwr', edgecolor='k')
plt.title('Linearly Separable Data') plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.show()
```



Plot the data for label, marker, color in zip(range(2), ('s', '^'), ('lightblue', 'orange')):

```
plt.scatter(X[y == label, 0], X[y == label, 1], c=color, marker=marker, edgecolor='k', label=f'Class
{label}')
```



```
from sklearn.linear_model import Perceptron
```

```

# Generate linearly separable data
X, y = make_classification(n_samples=100, n_features=2, n_informative=2, n_redundant=0,
n_clusters_per_class=1, class_sep=2, random_state=42)

# Create a DataFrame to simulate df
df = pd.DataFrame(X, columns=['Feature 1', 'Feature 2']) df['Label']
= y

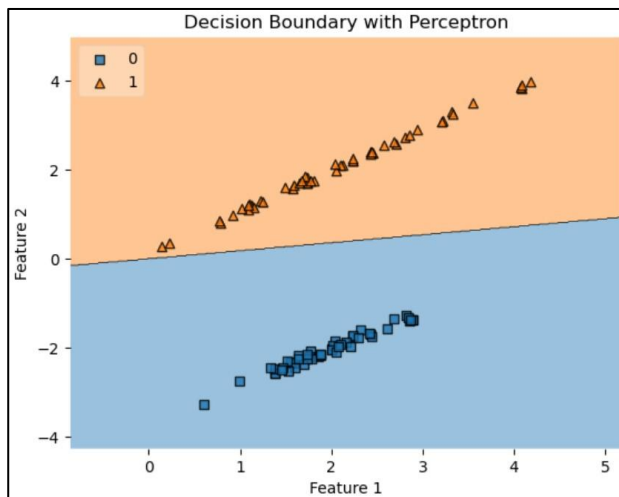
# Extract features and labels X
= df.iloc[:, [0, 1]].values
y = df.iloc[:, 2].values

# Train a Perceptron classifier clf
= Perceptron()
clf.fit(X, y)

# Plot decision regions
plot_decision_regions(X, y, clf=clf, legend=2)

# Add title and axis labels
plt.title('Decision Boundary with Perceptron')
plt.xlabel('Feature 1') plt.ylabel('Feature
2')
plt.show()

```



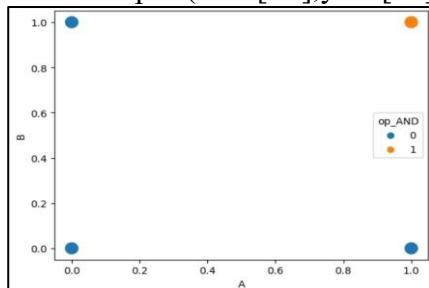
Signature: _____

b) Perceptron: Unable to capture XOR

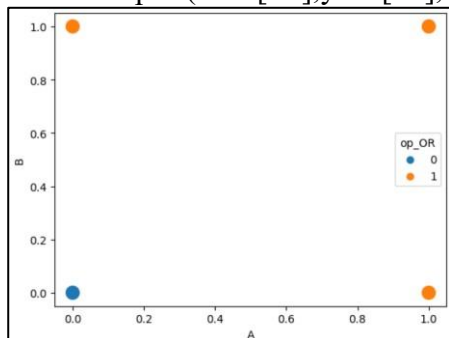
```
data = {  
    'A':[0,1,0,1],  
    'B':[0,0,1,1],  
    'op_AND':[0,0,0,1],  
    'op_OR':[0,1,1,1],  
    'op_XOR':[0,1,1,0]  
}  
df=pd.DataFrame(data)  
df
```

	A	B	op_AND	op_OR	op_XOR
0	0	0	0	0	0
1	1	0	0	1	1
2	0	1	0	1	1
3	1	1	1	1	0

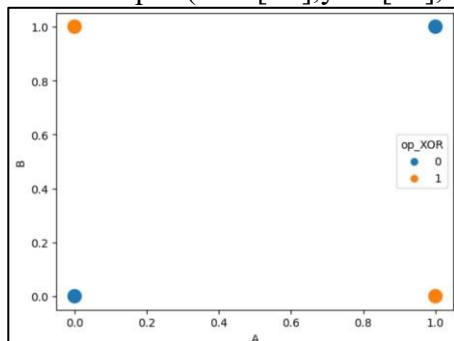
```
sns.scatterplot(x=df['A'],y=df['B'],hue=df['op_AND'],s=200)
```



```
sns.scatterplot(x=df['A'],y=df['B'],hue=df['op_OR'],s=200)
```



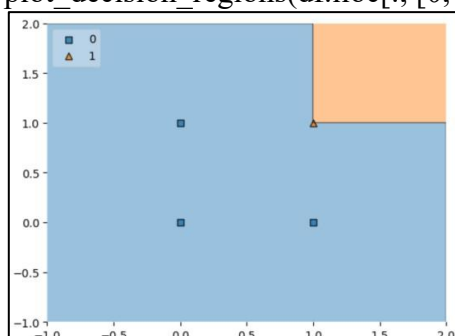
```
sns.scatterplot(x=df['A'],y=df['B'],hue=df['op_XOR'],s=200)
```



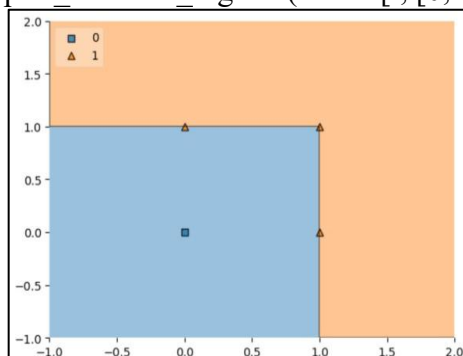
```
p1=Perceptron() p2=Perceptron()
p3=Perceptron()
p1.fit(df[['A','B']],df['op_AND'])
p2.fit(df[['A','B']],df['op_OR'])
p3.fit(df[['A','B']],df['op_XOR'])
```

```
▼ Perceptron
Perceptron()
```

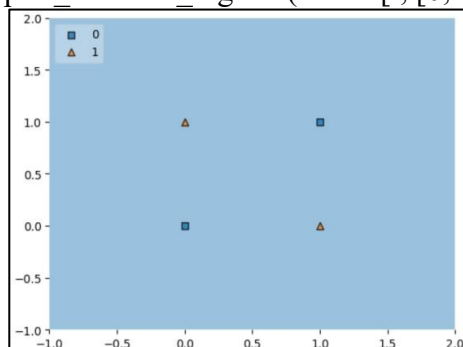
```
from mlxtend.plotting import plot_decision_regions
plot_decision_regions(df.iloc[:, [0, 1]].values,df.iloc[:,2].values,clf=p1,legend=2)
```



```
plot_decision_regions(df.iloc[:, [0, 1]].values,df2.iloc[:,3].values,clf=p2,legend=2)
```



```
plot_decision_regions(df.iloc[:, [0, 1]].values,df.iloc[:,4].values,clf=p3,legend=2)
```



Signature: _____

PRACTICAL NO.: 02 (a)

AIM: Customer Churn (Binary Classification using ANN)

```
import tensorflow as tf
import pandas as pd
import numpy as np
```

```
from tensorflow import keras
from tensorflow.keras import layers, Sequential
from tensorflow.keras.layers import Dense
```

```
df = pd.read_csv("/content/Churn_Modelling.csv")
df.head()
```

RowNumber	CustomerId	Surname	CreditScore	Geography	Gender	Age	Tenure	Balance	NumOfProducts	HasCrCard	IsActiveMember	EstimatedSalary
0	1	15634602	Hargrave	France	Female	42	2	0.00	1	1	1	101348.88
1	2	15647311	Hill	Spain	Female	41	1	83807.86	1	0	1	112542.58
2	3	15619304	Onio	France	Female	42	8	159660.80	3	1	0	113931.57
3	4	15701354	Boni	France	Female	39	1	0.00	2	0	0	93826.63
4	5	15737888	Mitchell	Spain	Female	43	2	125510.82	1	1	1	79084.10

```
df.isnull().sum()
```

```
RowNumber      0
CustomerId     0
Surname         0
CreditScore    0
Geography      0
Gender         0
Age            0
Tenure         0
Balance        0
NumOfProducts  0
HasCrCard      0
IsActiveMember 0
EstimatedSalary 0
Exited         0
dtype: int64
```

```
df.shape
```

```
(10000, 14)
```

```
df.columns
```

```
Index(['RowNumber', 'CustomerId', 'Surname', 'CreditScore', 'Geography',
       'Gender', 'Age', 'Tenure', 'Balance', 'NumOfProducts', 'HasCrCard',
       'IsActiveMember', 'EstimatedSalary', 'Exited'],
      dtype='object')
```

```
df.drop(['RowNumber', 'CustomerId', 'Surname'], axis=1, inplace=True)
df.head()
```

	CreditScore	Geography	Gender	Age	Tenure	Balance	NumOfProducts	HasCrCard	IsActiveMember	EstimatedSalary	Exited
0	619	France	Female	42	2	0.00	1	1	1	101348.88	1
1	608	Spain	Female	41	1	83807.86	1	0	1	112542.58	0
2	502	France	Female	42	8	159660.80	3	1	0	113931.57	1
3	699	France	Female	39	1	0.00	2	0	0	93826.63	0
4	850	Spain	Female	43	2	125510.82	1	1	1	79084.10	0

```
df['Gender'].value_counts()
```

```
Gender
Male      5457
Female    4543
Name: count, dtype: int64
```

```
df["Geography"].value_counts()
```

```
Geography
France    5014
Germany   2509
Spain     2477
Name: count, dtype: int64
```

```
df = pd.get_dummies(df, columns=['Geography', 'Gender'], dtype= int ) df.head()
```

	CreditScore	Age	Tenure	Balance	NumOfProducts	HasCrCard	IsActiveMember	EstimatedSalary	Exited	Geography_France	Geography_Germany
0	619	42	2	0.00	1	1	1	101348.88	1	1	0
1	608	41	1	83807.86	1	0	1	112542.58	0	0	0
2	502	42	8	159660.80	3	1	0	113931.57	1	1	0
3	699	39	1	0.00	2	0	0	93826.63	0	1	0
4	850	43	2	125510.82	1	1	1	79084.10	0	0	0

```
x = df.drop('Exited', axis=1) y
= df['Exited']
```

```
from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=0)
```

```
from sklearn.preprocessing import StandardScaler
sc = StandardScaler() x_train =
sc.fit_transform(x_train)
x_test = sc.transform(x_test)
```

```
model = Sequential()
model.add(Dense(units=3, activation='sigmoid', input_dim=13))
model.add(Dense(units=2, activation='sigmoid')) model.add(Dense(units=1,
activation='sigmoid'))
```

```
model.summary()
```

```
Model: "sequential_1"
Layer (type)                 Output Shape              Param #
-----
dense_3 (Dense)              (None, 3)                 42
dense_4 (Dense)              (None, 2)                 8
dense_5 (Dense)              (None, 1)                 3
-----
Total params: 53 (212.00 Byte)
Trainable params: 53 (212.00 Byte)
Non-trainable params: 0 (0.00 Byte)
```

```
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
model.layers[0].get_weights()
```

```
[array([[ 0.27477473,  0.58317274,  0.45780545],
        [ 0.34762233,  0.5840948 , -0.20378408],
        [-0.2972454 ,  0.39887244,  0.6027469 ],
        [-0.35907713, -0.13817823, -0.58515453],
        [-0.06439942, -0.26999915, -0.40960294],
        [ 0.01912552, -0.32841358, -0.38925815],
        [-0.5506319 , -0.28398386,  0.35141075],
        [ 0.3447131 ,  0.07300341,  0.23043859],
        [ 0.3491555 , -0.47870082, -0.49032572],
        [-0.24806485, -0.5903485 , -0.54736537],
        [-0.37863892, -0.23410189,  0.21590507],
        [ 0.4092074 ,  0.01194042, -0.5175006 ],
        [ 0.22989446, -0.10703623,  0.5810409 ]]), dtype=float32),
 array([0., 0., 0.], dtype=float32)]
```

```
model.layers[1].get_weights()
```

```
[array([[ 0.0024811 ,  0.50590825],
        [-0.2190789 , -0.2797523 ]],
      dtype=float32),
 array([0., 0.], dtype=float32)]
```

```
model.fit(x_train, y_train, batch_size=50, epochs=10, verbose=1, validation_split=0.2)
```

```
Epoch 1/10
128/128 [=====] - 3s 9ms/step - loss: 0.6667 - accuracy: 0.6975 - val_loss: 0.6299 - val_accuracy: 0.7969
Epoch 2/10
128/128 [=====] - 0s 3ms/step - loss: 0.6019 - accuracy: 0.7958 - val_loss: 0.5763 - val_accuracy: 0.7969
Epoch 3/10
128/128 [=====] - 0s 3ms/step - loss: 0.5581 - accuracy: 0.7958 - val_loss: 0.5403 - val_accuracy: 0.7969
Epoch 4/10
128/128 [=====] - 0s 3ms/step - loss: 0.5297 - accuracy: 0.7958 - val_loss: 0.5180 - val_accuracy: 0.7969
Epoch 5/10
128/128 [=====] - 0s 3ms/step - loss: 0.5127 - accuracy: 0.7958 - val_loss: 0.5053 - val_accuracy: 0.7969
Epoch 6/10
128/128 [=====] - 0s 3ms/step - loss: 0.5030 - accuracy: 0.7958 - val_loss: 0.4980 - val_accuracy: 0.7969
Epoch 7/10
128/128 [=====] - 0s 3ms/step - loss: 0.4972 - accuracy: 0.7958 - val_loss: 0.4934 - val_accuracy: 0.7969
Epoch 8/10
128/128 [=====] - 0s 3ms/step - loss: 0.4930 - accuracy: 0.7958 - val_loss: 0.4897 - val_accuracy: 0.7969
Epoch 9/10
128/128 [=====] - 0s 3ms/step - loss: 0.4893 - accuracy: 0.7958 - val_loss: 0.4863 - val_accuracy: 0.7969
Epoch 10/10
128/128 [=====] - 0s 3ms/step - loss: 0.4856 - accuracy: 0.7958 - val_loss: 0.4828 - val_accuracy: 0.7969
<keras.src.callbacks.History at 0x7d6f5c6e0280>
```

```
y_pred = model.predict(x_test)
```

```
63/63 [=====] - 0s 2ms/step
```

```
y_pred
```

```
array([[0.2562232 ],
       [0.23548149],
       [0.21577395],
       ...,
       [0.20127527],
       [0.21407473],
       [0.24438597]], dtype=float32)
```

```
y_pred = y_pred.argmax(axis=-1)
```

```
from sklearn.metrics import accuracy_score
accuracy_score(y_test, y_pred)
```

```
0.7975
```

Signature: _____

PRACTICAL NO.: 02 (b)

AIM: MNIST Dataset (Multiclass Classification using ANN)

```
import tensorflow as tf
import pandas as pd
import numpy as np
```

```
from tensorflow import keras
from tensorflow.keras import layers, Sequential
from tensorflow.keras.layers import Dense
train_df = pd.read_csv("/content/train.csv")
train_df.head()
```

	label	pixel0	pixel1	pixel2	pixel3	pixel4	pixel5	pixel6	pixel7	pixel8	...	pixel774	pixel775	pixel776	pixel777	pixel778	pixel779
0	1	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0
2	1	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0
3	4	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0
4	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0

5 rows x 785 columns

```
train_df.isnull().sum()
```

```
label      0
pixel0     0
pixel1     0
pixel2     0
pixel3     0
...
pixel779   0
pixel780   0
pixel781   0
pixel782   0
pixel783   0
Length: 785, dtype: int64
```

```
train_df.shape
```

```
(42000, 785)
```

```
x = train_df.drop('label', axis=1)
y = train_df['label']
```

```
from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=0)
```

```
from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
x_train = sc.fit_transform(x_train)
x_test = sc.transform(x_test)
```

```
model = Sequential()
model.add(Dense(units=128, activation='relu', input_dim=784))
model.add(Dense(units=80, activation='relu'))
model.add(Dense(units=10, activation='softmax'))
```

```
Model: "sequential_7"

Layer (type)                Output Shape                Param #
=====
dense_21 (Dense)            (None, 128)                100480
dense_22 (Dense)            (None, 80)                  10320
dense_23 (Dense)            (None, 10)                  810
=====
Total params: 111610 (435.98 KB)
Trainable params: 111610 (435.98 KB)
Non-trainable params: 0 (0.00 Byte)
```

```
model.layers[0].get_weights()
```

[illegible]

```
[array([[ -0.15624149,   0.02537861,  -0.06395023, ...,  -0.05318847,  
         -0.00713874,   0.10087174],  
       [ -0.04413277,  -0.00589547,  -0.0791691 , ...,  -0.12143775,  
        -0.15048888,   0.01031816],  
       [ -0.01764677,   0.06007096,  -0.16320327, ...,   0.07837842,  
        -0.15495947,  -0.00963637],  
       ...,  
       [ -0.06500816,  -0.13640904,   0.10746846, ...,  -0.03007014,  
        -0.14111118,   0.07219456],  
       [  0.11114174,  -0.16976587,  -0.0160839 , ...,  -0.05917796,  
         0.06286505,  -0.09894761],  
       [  0.03663093,   0.01428761,   0.05698816, ...,   0.00331713,  
        -0.16017482,  -0.02507338]], dtype=float32),  
 array([0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.,  
        0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.,  
        0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.,  
        0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.,  
        0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.,  
        0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.]
```

12

```
Epoch 1/10
538/538 [=====] - 4s 6ms/step - loss: 0.3137 - accuracy: 0.9049 - val_loss: 0.1846 - val_accuracy: 0.9452
Epoch 2/10
538/538 [=====] - 2s 4ms/step - loss: 0.1106 - accuracy: 0.9651 - val_loss: 0.1636 - val_accuracy: 0.9537
Epoch 3/10
538/538 [=====] - 2s 4ms/step - loss: 0.0667 - accuracy: 0.9791 - val_loss: 0.1714 - val_accuracy: 0.9545
Epoch 4/10
538/538 [=====] - 2s 4ms/step - loss: 0.0469 - accuracy: 0.9850 - val_loss: 0.1620 - val_accuracy: 0.9609
Epoch 5/10
538/538 [=====] - 2s 4ms/step - loss: 0.0355 - accuracy: 0.9884 - val_loss: 0.1600 - val_accuracy: 0.9628
Epoch 6/10
538/538 [=====] - 4s 8ms/step - loss: 0.0239 - accuracy: 0.9922 - val_loss: 0.1929 - val_accuracy: 0.9604
Epoch 7/10
```

```
y_pred = model.predict(x_test)
```

```
263/263 [=====] - 1s 2ms/step
```

```
y_pred
```

```
array([[1.4865213e-07, 2.0831956e-09, 5.3478393e-06, ..., 7.5377642e-08,
        1.2525517e-03, 1.4948829e-04],
       [7.4322262e-14, 2.2606442e-17, 6.6010348e-14, ..., 1.3589097e-11,
        2.3813153e-11, 4.2938424e-11],
       [1.3230222e-10, 1.6269179e-20, 6.4765638e-15, ..., 4.4112735e-06,
        1.1426775e-08, 9.9999553e-01],
       ...,
       [6.4957627e-12, 1.5501279e-10, 9.9998230e-01, ..., 8.6845491e-09,
        1.7701408e-05, 4.5676185e-13],
       [1.2812490e-07, 3.3971326e-15, 2.3855668e-08, ..., 9.9973762e-01,
        5.4589323e-11, 2.6233809e-04],
       [2.0888731e-09, 3.4155602e-12, 9.9999982e-01, ..., 6.4930422e-11,
        7.6808471e-16, 3.3484316e-15]], dtype=float32)
```

```
y_pred = y_pred.argmax(axis=-1)
```

```
0.9596428571428571
```

Signature: _____

PRACTICAL NO.: 03

AIM: Regression problem using ANN

```
import tensorflow as tf
import pandas as pd
import numpy as np
```

```
from tensorflow import keras
from tensorflow.keras import layers, Sequential
from tensorflow.keras.layers import Dense
from sklearn.preprocessing import StandardScaler
from sklearn.preprocessing import MinMaxScaler
```

```
df = pd.read_csv("/content/Admission_Predict_Ver1.1 - Admission_Predict_Ver1.1.csv")
df.head()
```

	Serial No.	GRE Score	TOEFL Score	University Rating	SOP	LOR	CGPA	Research	Chance of Admit
0	1	337	118	4	4.5	4.5	9.65	1	0.92
1	2	324	107	4	4.0	4.5	8.87	1	0.76
2	3	316	104	3	3.0	3.5	8.00	1	0.72
3	4	322	110	3	3.5	2.5	8.67	1	0.80
4	5	314	103	2	2.0	3.0	8.21	0	0.65

```
df.isnull().sum()
```

```
Serial No.      0
GRE Score       0
TOEFL Score     0
University Rating 0
SOP             0
LOR            0
CGPA           0
Research        0
Chance of Admit 0
dtype: int64
```

```
df.drop(['Serial No.'], axis=1, inplace=True)
df.head()
```

	GRE Score	TOEFL Score	University Rating	SOP	LOR	CGPA	Research	Chance of Admit
0	337	118	4	4.5	4.5	9.65	1	0.92
1	324	107	4	4.0	4.5	8.87	1	0.76
2	316	104	3	3.0	3.5	8.00	1	0.72
3	322	110	3	3.5	2.5	8.67	1	0.80
4	314	103	2	2.0	3.0	8.21	0	0.65

```
df.columns
```

```
Index(['GRE Score', 'TOEFL Score', 'University Rating', 'SOP', 'LOR', 'CGPA',
      'Research', 'Chance of Admit'],
      dtype='object')
```

```
df.shape
```

```
(500, 8)
```

```
x = df.iloc[:,0:7]
```

```
y = df['Chance of Admit']
```

X

	GRE Score	TOEFL Score	University Rating	SOP	LOR	CGPA	Research
0	337	118	4	4.5	4.5	9.65	1
1	324	107	4	4.0	4.5	8.87	1
2	316	104	3	3.0	3.5	8.00	1
3	322	110	3	3.5	2.5	8.67	1
4	314	103	2	2.0	3.0	8.21	0
...	...	...	...	...	...	...	...
495	332	108	5	4.5	4.0	9.02	1
496	337	117	5	5.0	5.0	9.87	1
497	330	120	5	4.5	5.0	9.56	1
498	312	103	4	4.0	5.0	8.43	0
499	327	113	4	4.5	4.5	9.04	0

500 rows × 7 columns

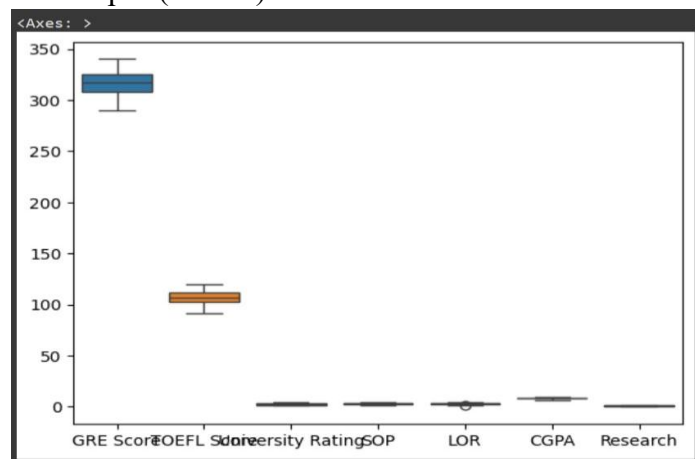
y

```
0      0.92
1      0.76
2      0.72
3      0.80
4      0.65
...
495    0.87
496    0.96
497    0.93
498    0.73
499    0.84
Name: Chance of Admit, Length: 500, dtype: float64
```

```
from sklearn.model_selection import train_test_split
```

```
x_train, x_test, y_train, y_test = train_test_split(x,y, test_size=0.2, random_state=42)
```

```
sns.boxplot(data=x)
```



```
x_train.shape
```

```
(400, 7)
```

```
x_test.shape
```

```
(100, 7)
```

```
#minmax scalar
```

```
#only scale x, Reason: y we have to predict
```

```
scalar = MinMaxScaler() x_train =
```

```
scalar.fit_transform(x_train)
```

```
x_test = scalar.transform(x_test)
```

```
model = Sequential()
```

```
model.add(Dense(7, activation='relu',input_dim=7)) model.add(Dense(1,  
activation='linear'))
```

```
model.compile(optimizer='adam', loss='mean_squared_error',metrics=['accuracy']) model.fit(x_train,  
y_train, epochs=100, batch_size=10,verbose=1)
```

```
Epoch 1/100  
40/40 [=====] - 1s 2ms/step - loss: 0.1177 - a  
Epoch 2/100  
40/40 [=====] - 0s 2ms/step - loss: 0.0519 - a  
Epoch 3/100  
40/40 [=====] - 0s 2ms/step - loss: 0.0435 - a  
Epoch 4/100  
40/40 [=====] - 0s 2ms/step - loss: 0.0387 - a  
Epoch 5/100  
40/40 [=====] - 0s 2ms/step - loss: 0.0349 - a  
Epoch 6/100  
40/40 [=====] - 0s 2ms/step - loss: 0.0315 - a  
Epoch 7/100  
40/40 [=====] - 0s 2ms/step - loss: 0.0296 - a  
Epoch 8/100  
40/40 [=====] - 0s 2ms/step - loss: 0.0272 - a  
Epoch 9/100
```

```
model.summary()
```

```
Model: "sequential_4"  


| Layer (type)     | Output Shape | Param # |
|------------------|--------------|---------|
| dense_10 (Dense) | (None, 7)    | 56      |
| dense_11 (Dense) | (None, 1)    | 8       |

  
Total params: 64 (256.00 Byte)  
Trainable params: 64 (256.00 Byte)  
Non-trainable params: 0 (0.00 Byte)
```

```
y_pred = model.predict(x_test)
```

```
from sklearn.metrics import mean_squared_error,mean_absolute_error,r2_score
```

```
print(r2_score(y_test, y_pred))
```

```
0.8053834364621935
```

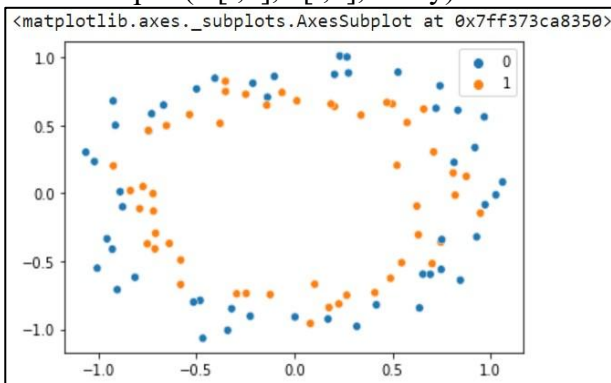
Signature: _____

PRACTICAL NO.: 04

AIM: Implementation of Regularization Technique : Early Stopping

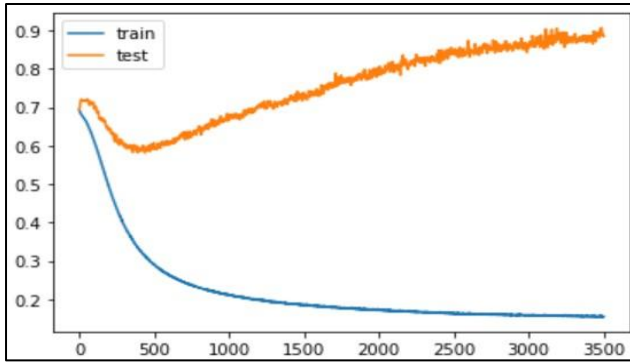
```
import tensorflow as tf
import numpy as np
import pandas as pd
from pylab import rcParams
import matplotlib.pyplot as plt
import warnings
from mlxtend.plotting import plot_decision_regions
from matplotlib.colors import ListedColormap
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dropout, Dense
from tensorflow.keras.callbacks import EarlyStopping
from sklearn.model_selection import train_test_split
from sklearn.datasets import make_circles
import seaborn as sns
```

```
X, y = make_circles(n_samples=100, noise=0.1, random_state=1)
sns.scatterplot(X[:,0],X[:,1],hue=y)
```

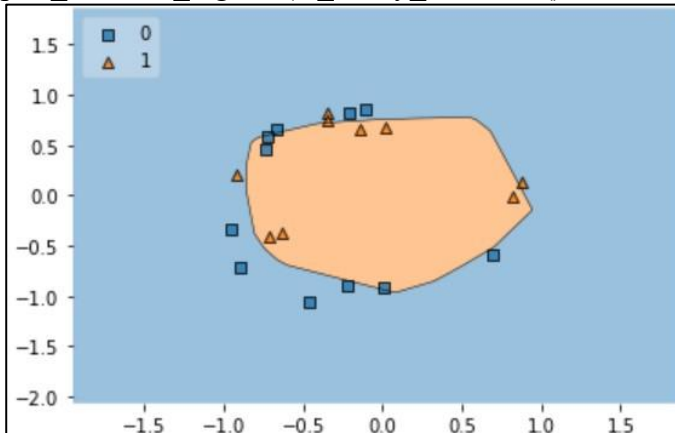


```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20, random_state=2)
model = Sequential()
```

```
model.add(Dense(256, input_dim=2, activation='relu'))
model.add(Dense(1, activation='sigmoid'))
model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
history = model.fit(X_train, y_train, validation_data=(X_test, y_test), epochs=3500, verbose=0)
plt.plot(history.history['loss'], label='train')
plt.plot(history.history['val_loss'], label='test')
plt.legend()
plt.show()
```



```
plot_decision_regions(X_test, y_test.ravel(), clf = model, legend=2) plt.show()
```



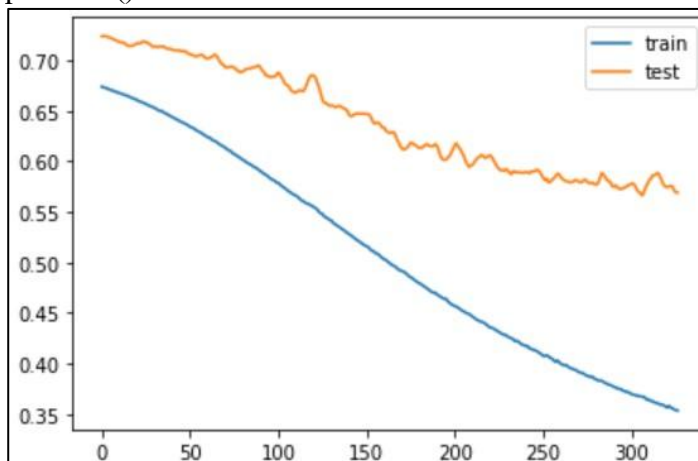
#Early Stopping

```
model = Sequential()
model.add(Dense(256, input_dim=2, activation='relu')) model.add(Dense(1,
activation='sigmoid'))
model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
callback = EarlyStopping( monitor="val_loss", min_delta=0.00001,
patience=20, verbose=1,

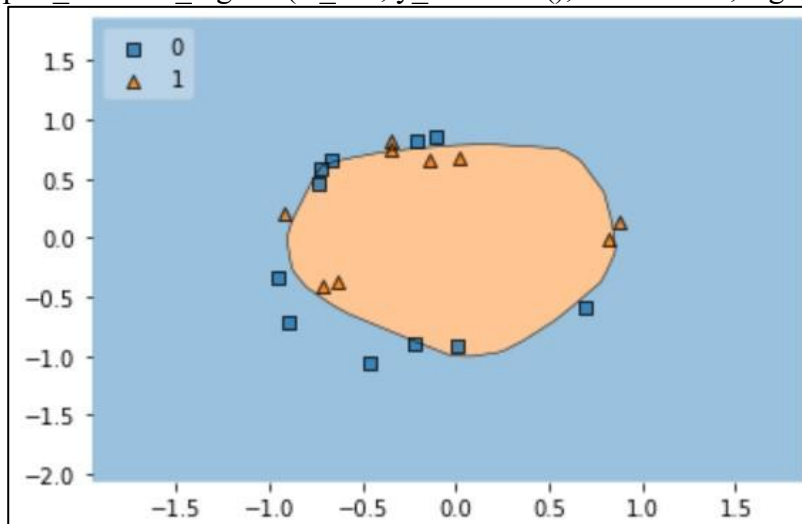
mode="auto",
baseline=None,
restore_best_weights=False
)
history = model.fit(X_train, y_train, validation_data=(X_test, y_test), epochs=3500, callbacks=callback)
```

```
Epoch 1/3500
3/3 [=====] - 0s 27ms/step - loss: 0.6737 - accuracy: 0.5750 - val_loss: 0.7236 - val_accuracy: 0.4000
Epoch 2/3500
3/3 [=====] - 0s 16ms/step - loss: 0.6733 - accuracy: 0.5750 - val_loss: 0.7238 - val_accuracy: 0.4000
Epoch 3/3500
3/3 [=====] - 0s 12ms/step - loss: 0.6728 - accuracy: 0.5750 - val_loss: 0.7240 - val_accuracy: 0.4000
Epoch 4/3500
3/3 [=====] - 0s 16ms/step - loss: 0.6720 - accuracy: 0.5750 - val_loss: 0.7234 - val_accuracy: 0.4000
Epoch 5/3500
3/3 [=====] - 0s 14ms/step - loss: 0.6712 - accuracy: 0.5625 - val_loss: 0.7230 - val_accuracy: 0.4000
Epoch 6/3500
3/3 [=====] - 0s 22ms/step - loss: 0.6706 - accuracy: 0.5750 - val_loss: 0.7221 - val_accuracy: 0.4000
Epoch 7/3500
3/3 [=====] - 0s 14ms/step - loss: 0.6701 - accuracy: 0.5750 - val_loss: 0.7213 - val_accuracy: 0.4000
Epoch 8/3500
3/3 [=====] - 0s 15ms/step - loss: 0.6693 - accuracy: 0.5750 - val_loss: 0.7205 - val_accuracy: 0.4000
Epoch 9/3500
3/3 [=====] - 0s 14ms/step - loss: 0.6687 - accuracy: 0.5750 - val_loss: 0.7198 - val_accuracy: 0.4000
Epoch 10/3500
3/3 [=====] - 0s 13ms/step - loss: 0.6681 - accuracy: 0.5875 - val_loss: 0.7187 - val_accuracy: 0.4000
Epoch 11/3500
3/3 [=====] - 0s 21ms/step - loss: 0.6678 - accuracy: 0.5875 - val_loss: 0.7180 - val_accuracy: 0.4000
Epoch 12/3500
3/3 [=====] - 0s 13ms/step - loss: 0.6669 - accuracy: 0.5875 - val_loss: 0.7178 - val_accuracy: 0.4000
Epoch 13/3500
```

```
plt.plot(history.history['loss'], label='train')
plt.plot(history.history['val_loss'], label='test') plt.legend()
plt.show()
```



```
plot_decision_regions(X_test, y_test.ravel(), clf = model, legend=2) plt.show()
```



Signature: _____

PRACTICAL NO.: 05

AIM: Implementation of Regularization technique: Dropout a) Classification

```
import numpy as np
import matplotlib.pyplot as plt

X = np.array([[-1.58986e-01, 4.23977e-01],
               [-3.47926e-01, 4.70760e-01],
               [-5.04608e-01, 3.53801e-01],
               [-5.96774e-01, 1.14035e-01],
               [-5.18433e-01, -1.72515e-01],
               [-2.92627e-01, -2.07602e-01],
               [-1.58986e-01, -4.38596e-02],
               [-5.76037e-02, 1.43275e-01],
               [-7.14286e-02, 2.71930e-01],
               [-2.97235e-01, 3.47953e-01],
               [-4.17051e-01, 2.01754e-01],
               [-4.40092e-01, 8.77193e-03],
               [-3.24885e-01, -3.21637e-02],
               [-2.46544e-01, 5.55556e-02],
               [-2.18894e-01, 2.01754e-01],
               [-3.43318e-01, 1.60819e-01],
               [-5.09217e-01, 7.89474e-02],
               [-3.84793e-01, -9.06433e-02],
               [-1.49770e-01, 1.25731e-01],
               [-1.95853e-01, 3.24561e-01],
               [-3.91705e-02, -2.19298e-01],
               [-1.08295e-01, -3.01170e-01],
               [-1.86636e-01, -3.30409e-01],
               [-2.18894e-01, -4.23977e-01],
               [-8.06452e-02, -5.64327e-01],
               [ 6.68203e-02, -5.17544e-01],
               [ 9.44700e-02, -3.24561e-01],
               [ 1.86636e-01, -1.66667e-01],
               [ 6.22120e-02, -7.30994e-02],
               [ 2.07373e-02, -1.95906e-01],
               [ 2.99539e-02, -3.42105e-01],
               [-9.90783e-02, -3.77193e-01],
               [-6.91244e-03, -4.64912e-01],
               [ 1.31336e-01, -4.29825e-01],
               [ 2.32719e-01, -1.95906e-01],
               [ 8.52535e-02, -8.47953e-02],
               [-1.31336e-01, -2.36842e-01],
               [ 2.30415e-03, -1.25731e-01],
               [ 1.22120e-01, -2.92398e-03]])

y = np.array([1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1.,
```

```

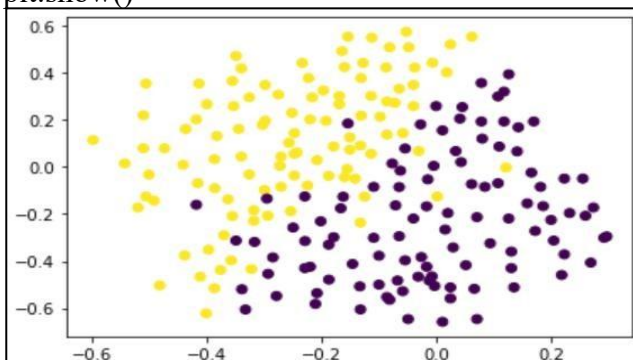
1., 1., 1., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.,
0., 0., 1., 1., 1., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.,
0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.,
0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 1., 1., 1., 1.,    1.,
1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1.,    1., 1.,
1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 0., 0.,    0., 0., 1.,
1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1.,    1., 1., 1., 1.,
1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 0., 0.,    0., 0., 0., 0., 0.,
0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.,    0., 0., 0., 0., 0.,
0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.,
0., 0., 0., 0., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1.,
0., 1., 1., 1., 1., 0., 0.])

```

```

import matplotlib.pyplot as plt
plt.scatter(X[:,0], X[:,1], c=y)
plt.show()

```



```

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.layers import Dropout
from tensorflow.keras.optimizers import Adam
model = Sequential()

```

```

model.add(Dense(128, input_dim=2, activation="relu"))
model.add(Dense(128, activation="relu"))
model.add(Dense(1, activation="sigmoid"))

```

```

adam = Adam(learning_rate=0.01)
model.compile(loss='binary_crossentropy', optimizer=adam, metrics=['accuracy'])

```

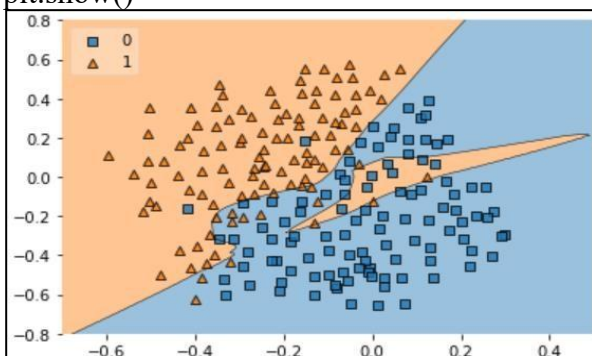
```

history = model.fit(X, y, epochs=500, validation_split = 0.2, verbose=1)

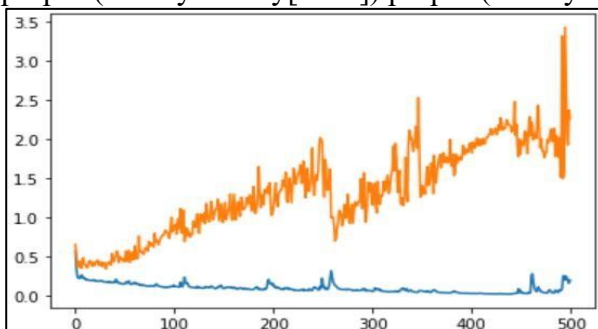
```

```
Epoch 1/500
6/6 [=====] - 1s 36ms/step - loss: 0.5667 - accuracy: 0.8393 - val_loss: 0.6492 - val_accuracy: 0.6279
Epoch 2/500
6/6 [=====] - 0s 9ms/step - loss: 0.3443 - accuracy: 0.8393 - val_loss: 0.5098 - val_accuracy: 0.6977
Epoch 3/500
6/6 [=====] - 0s 12ms/step - loss: 0.2406 - accuracy: 0.8929 - val_loss: 0.3613 - val_accuracy: 0.8605
Epoch 4/500
6/6 [=====] - 0s 8ms/step - loss: 0.2168 - accuracy: 0.9167 - val_loss: 0.3645 - val_accuracy: 0.8605
Epoch 5/500
6/6 [=====] - 0s 8ms/step - loss: 0.2301 - accuracy: 0.8988 - val_loss: 0.4342 - val_accuracy: 0.8605
Epoch 6/500
6/6 [=====] - 0s 9ms/step - loss: 0.2191 - accuracy: 0.9167 - val_loss: 0.4084 - val_accuracy: 0.8605
Epoch 7/500
6/6 [=====] - 0s 8ms/step - loss: 0.2567 - accuracy: 0.8869 - val_loss: 0.3422 - val_accuracy: 0.8605
Epoch 8/500
6/6 [=====] - 0s 8ms/step - loss: 0.2513 - accuracy: 0.8988 - val_loss: 0.4096 - val_accuracy: 0.8372
Epoch 9/500
6/6 [=====] - 0s 8ms/step - loss: 0.2139 - accuracy: 0.9107 - val_loss: 0.4799 - val_accuracy: 0.7209
Epoch 10/500
6/6 [=====] - 0s 12ms/step - loss: 0.2231 - accuracy: 0.9107 - val_loss: 0.3972 - val_accuracy: 0.8605
Epoch 11/500
6/6 [=====] - 0s 13ms/step - loss: 0.2033 - accuracy: 0.9226 - val_loss: 0.3831 - val_accuracy: 0.8605
Epoch 12/500
6/6 [=====] - 0s 11ms/step - loss: 0.1916 - accuracy: 0.9405 - val_loss: 0.3639 - val_accuracy: 0.8605
```

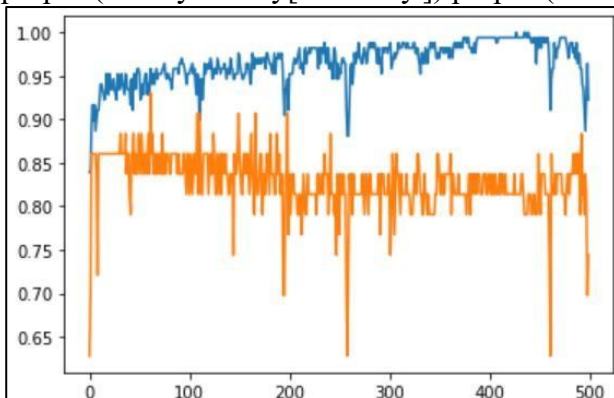
```
from mlxtend.plotting import plot_decision_regions
plot_decision_regions(X,
y.astype('int'), clf=model, legend=2)
plt.xlim(-0.7,0.5)
plt.ylim(-0.8,0.8)
plt.show()
```



```
plt.plot(history.history['loss']) plt.plot(history.history['val_loss'])
```



```
plt.plot(history.history['accuracy']) plt.plot(history.history['val_accuracy'])
```



```
model = Sequential()
```

```

model.add(Dense(128, input_dim=2, activation="relu"))
model.add(Dropout(0.5)) model.add(Dense(128,
activation="relu")) model.add(Dropout(0.5))
model.add(Dense(1, activation="sigmoid"))

```

```

adam = Adam(learning_rate=0.01) model.compile(loss='binary_crossentropy', optimizer=adam,
metrics=['accuracy'])

```

```

history = model.fit(X, y, epochs=500, validation_split = 0.2, verbose=1)

```

```

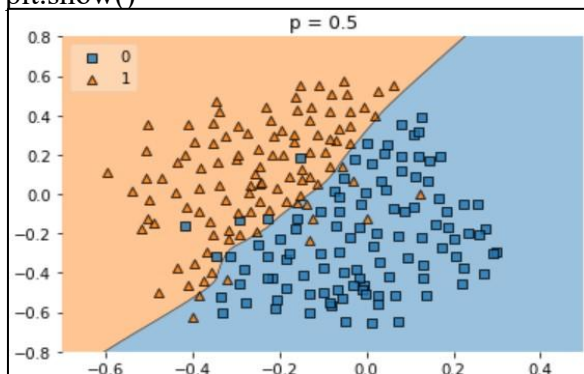
Epoch 1/500
6/6 [=====] - 1s 78ms/step - loss: 0.5846 - accuracy: 0.7738 - val_loss: 0.6484 - val_accuracy: 0.5814
Epoch 2/500
6/6 [=====] - 0s 6ms/step - loss: 0.3438 - accuracy: 0.8571 - val_loss: 0.9476 - val_accuracy: 0.5814
Epoch 3/500
6/6 [=====] - 0s 7ms/step - loss: 0.2886 - accuracy: 0.8631 - val_loss: 0.6692 - val_accuracy: 0.6977
Epoch 4/500
6/6 [=====] - 0s 6ms/step - loss: 0.2981 - accuracy: 0.8810 - val_loss: 0.4833 - val_accuracy: 0.7442
Epoch 5/500
6/6 [=====] - 0s 8ms/step - loss: 0.2408 - accuracy: 0.9167 - val_loss: 0.3649 - val_accuracy: 0.8140
Epoch 6/500
6/6 [=====] - 0s 6ms/step - loss: 0.2541 - accuracy: 0.9107 - val_loss: 0.3456 - val_accuracy: 0.8605
Epoch 7/500
6/6 [=====] - 0s 7ms/step - loss: 0.2637 - accuracy: 0.8988 - val_loss: 0.3714 - val_accuracy: 0.8372
Epoch 8/500
6/6 [=====] - 0s 8ms/step - loss: 0.2515 - accuracy: 0.8810 - val_loss: 0.4228 - val_accuracy: 0.8140
Epoch 9/500
6/6 [=====] - 0s 11ms/step - loss: 0.2342 - accuracy: 0.8929 - val_loss: 0.4598 - val_accuracy: 0.7907
Epoch 10/500
6/6 [=====] - 0s 6ms/step - loss: 0.2535 - accuracy: 0.8929 - val_loss: 0.4082 - val_accuracy: 0.8372
Epoch 11/500
6/6 [=====] - 0s 8ms/step - loss: 0.2290 - accuracy: 0.9286 - val_loss: 0.3471 - val_accuracy: 0.8605
Epoch 12/500
6/6 [=====] - 0s 6ms/step - loss: 0.2100 - accuracy: 0.9226 - val_loss: 0.3436 - val_accuracy: 0.8605
Epoch 13/500
6/6 [=====] - 0s 8ms/step - loss: 0.2578 - accuracy: 0.8929 - val_loss: 0.3524 - val_accuracy: 0.8605
Epoch 14/500
6/6 [=====] - 0s 8ms/step - loss: 0.2351 - accuracy: 0.8929 - val_loss: 0.3733 - val_accuracy: 0.8372

```

```

plot_decision_regions(X, y.astype('int'), clf=model, legend=2)
plt.xlim(-0.7,0.5) plt.ylim(-0.8,0.8)
plt.title('p = 0.5')
plt.show()

```



```

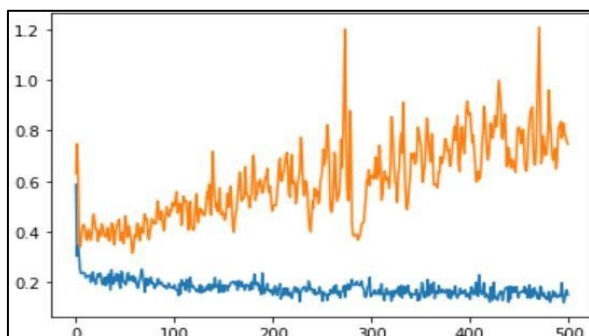
plt.plot(history.history['loss'])

```

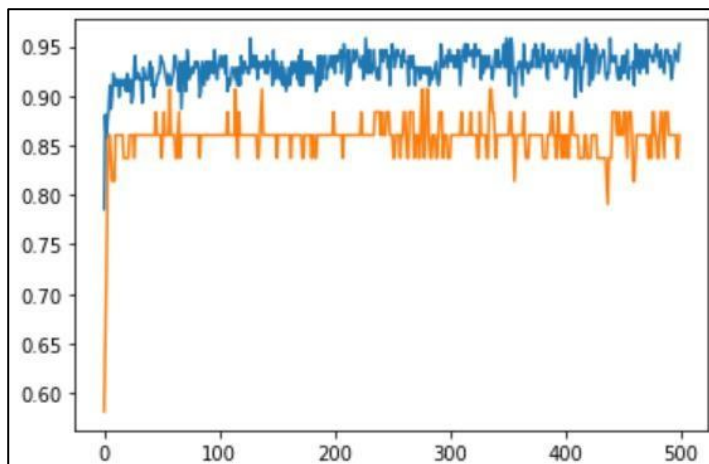
```

plt.plot(history.history['val_loss'])

```



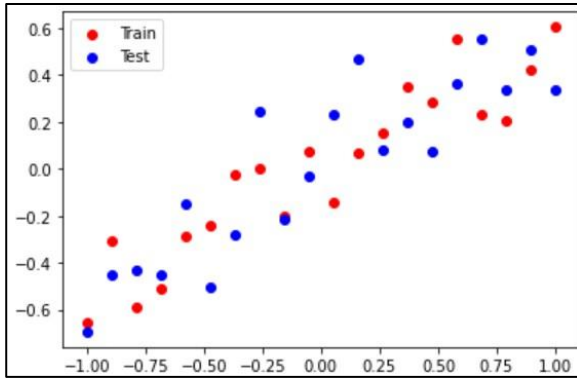
```
plt.plot(history.history['accuracy']) plt.plot(history.history['val_accuracy'])
```



b) Dropouts for Regression problem

```
import tensorflow as tf import numpy as np import
matplotlib.pyplot as plt from
tensorflow.keras.models import Sequential from
tensorflow.keras.layers import Dense from
tensorflow.keras.layers import Dropout from
tensorflow.keras.optimizers import Adam from
sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error
```

```
X_train = np.linspace(-1, 1, 20) y_train = np.array([-0.6561 , -
0.3099 , -0.59035, -0.50855, -0.285 ,
-0.2443 , -0.02445, 0.00135, -0.2006 , 0.07475,
-0.1422 , 0.06515, 0.15265, 0.3521 , 0.28415,
0.5524 , 0.23115, 0.20835, 0.4211, 0.60485]) X_test
= np.linspace(-1, 1, 20)
y_test = np.array([-0.69415, -0.451 , -0.43005, -0.4484 , -0.1475 ,
-0.5019 , -0.28055, 0.24595, -0.21425, -0.0286 ,
0.23415, 0.46575, 0.07955, 0.1973 , 0.0719 ,
0.3639 , 0.5536 , 0.3365 , 0.50705, 0.33435])
plt.scatter(X_train, y_train, c='red', label='Train')
plt.scatter(X_test, y_test, c='blue', label='Test') plt.legend()
plt.show()
```



```

model_1 = Sequential()
model_1.add(Dense(128, input_dim=1, activation="relu"))
model_1.add(Dense(128, activation="relu"))
model_1.add(Dense(1, activation="linear")) adam =
Adam(learning_rate=0.01)
model_1.compile(loss='mse', optimizer=adam, metrics=['mse'])
history = model_1.fit(X_train, y_train, epochs=500,
validation_data = (X_test, y_test), verbose=False)
# evaluate the model
_, train_mse = model_1.evaluate(X_train, y_train, verbose=0)
test_mse = model_1.evaluate(X_test, y_test, verbose=0)
print('Train: {}, Test: {}'.format(train_mse, test_mse))

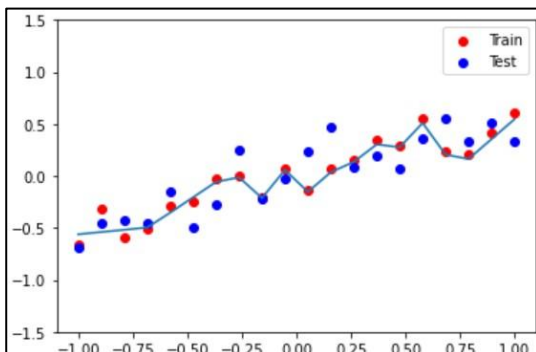
```

Train: 0.004562994930893183, Test: 0.04608117789030075

```

y_pred_1 = model_1.predict(X_test) plt.figure()
plt.scatter(X_train, y_train, c='red', label='Train')
plt.scatter(X_test, y_test, c='blue', label='Test')
plt.plot(X_test, y_pred_1)
plt.legend() plt.ylim((-1.5,
1.5))
plt.show()

```



#DropOut Model

```

model_2 = Sequential()
model_2.add(Dense(128, input_dim=1, activation="relu")) model_2.add(Dropout(0.2))

```

```

model_2.add(Dense(128, activation="relu")) model_2.add(Dropout(0.2))
model_2.add(Dense(1, activation="linear")) adam =
Adam(learning_rate=0.01) model_2.compile(loss='mse',
optimizer=adam, metrics=['mse'])
drop_out_history = model_2.fit(X_train, y_train, epochs=500,
validation_data = (X_test, y_test),
verbose=False)
#evaluate the model
_, train_mse = model_2.evaluate(X_train, y_train, verbose=0) _,
test_mse = model_2.evaluate(X_test, y_test, verbose=0)
print('Train: {}, Test: {}'.format(train_mse, test_mse))

```

```

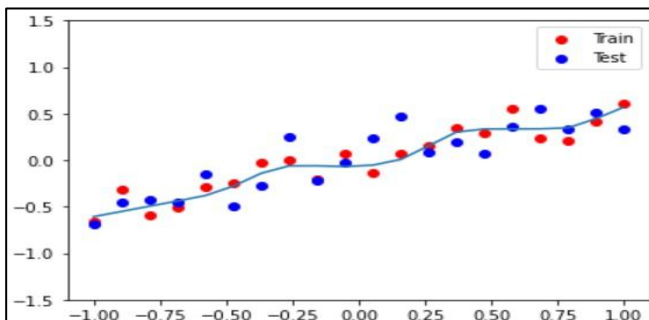
Train: 0.011907287873327732, Test: 0.03752660006284714

```

```

y_pred_2 = model_2.predict(X_test)
plt.figure() plt.scatter(X_train, y_train, c='red',
label='Train') plt.scatter(X_test, y_test, c='blue',
label='Test')
plt.plot(X_test, y_pred_2)
plt.legend() plt.ylim((-1.5,
1.5))
plt.show()

```



Signature: _____

PRACTICAL NO.: 06

AIM: Implementation of RNN

```
from keras.datasets import imdb from
keras.preprocessing.text import Tokenizer from
keras.preprocessing.sequence import pad_sequences from
keras import Sequential
from keras.layers import Dense, Embedding, Flatten, SimpleRNN
(x_train, y_train), (x_test, y_test) = imdb.load_data() x_train.shape
x_train[0]
```

```
[1,
14,
22,
16,
43,
530,
973,
1622,
1385,
65,
458,
4468,
66,
```

```
x_train = pad_sequences(x_train, padding="post", maxlen=50)
x_test = pad_sequences(x_test, padding="post", maxlen=50) model
= Sequential()
model.add(SimpleRNN(32,input_shape=(50,1),return_sequences=False))
model.add(Dense(1, activation="sigmoid")) model.summary()
```

Model: "sequential_2"

Layer (type)	Output Shape	Param #
simple_rnn_1 (SimpleRNN)	(None, 32)	1088
dense_1 (Dense)	(None, 1)	33

```
model.compile(optimizer="adam", loss="binary_crossentropy", metrics=["accuracy"])
model.fit(x_train, y_train, epochs=50, validation_data=[x_test, y_test])
```

```
Epoch 1/50
782/782 [=====] - 12s 14ms/step - loss: 0.6941 - accuracy: 0.5020 - val_loss: 0.6934 - val_accuracy: 0.5068
Epoch 2/50
782/782 [=====] - 14s 17ms/step - loss: 0.6932 - accuracy: 0.5072 - val_loss: 0.6936 - val_accuracy: 0.5072
Epoch 3/50
782/782 [=====] - 11s 15ms/step - loss: 0.6927 - accuracy: 0.5106 - val_loss: 0.6947 - val_accuracy: 0.5023
Epoch 4/50
782/782 [=====] - 11s 14ms/step - loss: 0.6928 - accuracy: 0.5121 - val_loss: 0.6941 - val_accuracy: 0.5064
Epoch 5/50
782/782 [=====] - 13s 17ms/step - loss: 0.6927 - accuracy: 0.5057 - val_loss: 0.6949 - val_accuracy: 0.5039
Epoch 6/50
782/782 [=====] - 11s 14ms/step - loss: 0.6925 - accuracy: 0.5090 - val_loss: 0.6935 - val_accuracy: 0.5041
Epoch 7/50
782/782 [=====] - 11s 14ms/step - loss: 0.6925 - accuracy: 0.5064 - val_loss: 0.6963 - val_accuracy: 0.5028
Epoch 8/50
```

Signature: _____

PRACTICAL NO.: 07

AIM: Implementation of CNN

#Get the api token created from Kaggle by using the user option present at right top corner

Go to Kaggle >> Profile >> Setting>> API>> create new token >> continue>> upload-download file in colab

Go to Kaggle >> pediatric-pneumonia-chest-x ray >> more>> copy API command >> paste it in colab
!kaggle datasets download -d andrewmvd/pediatric-pneumonia-chest-xray

```
Dataset URL: https://www.kaggle.com/datasets/andrewmvd/pediatric-pneumonia-chest-xray
License(s): Attribution 4.0 International (CC BY 4.0)
pediatric-pneumonia-chest-xray.zip: Skipping, found more recently modified local copy (use --force to force download)
```

```
import zipfile
zip_ref = zipfile.ZipFile('/content/pediatric-pneumonia-chest-xray.zip', 'r')
zip_ref.extractall('/content') zip_ref.close() import tensorflow as tf import
keras
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Conv2D, MaxPool2D, Flatten
#Generator
tr_data = keras.utils.image_dataset_from_directory(
directory = '/content/Pediatric Chest X-ray Pneumonia/train',
labels = 'inferred', label_mode = 'int', batch_size = 32,
image_size = (256,256)
)
```

```
Found 5232 files belonging to 2 classes.
```

```
test_data = keras.utils.image_dataset_from_directory(
directory = '/content/Pediatric Chest X-ray Pneumonia/test',
labels = 'inferred', label_mode = 'int', batch_size = 32,
image_size = (256,256)
)
```

```
Found 624 files belonging to 2 classes.
```

```
#normalizing images def
normalize(image, label):
    image = tf.cast(image/255. , tf.float32)
    return image, label
```

```
tr_data = tr_data.map(normalize)
test_data = test_data.map(normalize)
```

```
model = Sequential()
```

```

model.add(Conv2D(32,(3,3),activation='relu', input_shape=(256,256,3))) #32 filters of 3,3 shape
model.add(MaxPool2D((2,2), strides=2, padding='valid'))
model.add(Conv2D(64,(3,3),activation='relu', input_shape=(256,256,3)))
model.add(MaxPool2D((2,2), strides=2, padding='valid'))
model.add(Conv2D(128,(3,3),activation='relu', input_shape=(256,256,3)))
model.add(MaxPool2D((2,2), strides=2, padding='valid'))
model.add(Flatten())
model.add(Dense(128, activation='relu')) #the input_dim should be same as model.add(Dense(64,
activation='relu'))
model.add(Dense(1, activation='sigmoid'))
model.summary()

```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
conv2d_3 (Conv2D)	(None, 254, 254, 32)	896
max_pooling2d_3 (MaxPooling2D)	(None, 127, 127, 32)	0
conv2d_4 (Conv2D)	(None, 125, 125, 64)	18,496
max_pooling2d_4 (MaxPooling2D)	(None, 62, 62, 64)	0
conv2d_5 (Conv2D)	(None, 60, 60, 128)	73,856
max_pooling2d_5 (MaxPooling2D)	(None, 30, 30, 128)	0
flatten_1 (Flatten)	(None, 115200)	0
dense_3 (Dense)	(None, 128)	14,745,728
dense_4 (Dense)	(None, 64)	8,256
dense_5 (Dense)	(None, 1)	65

Total params: 14,847,297 (56.64 MB)
Trainable params: 14,847,297 (56.64 MB)
Non-trainable params: 0 (0.00 B)

```

model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy']) model.fit(tr_data,
epochs=10, validation_data=test_data)

```

```

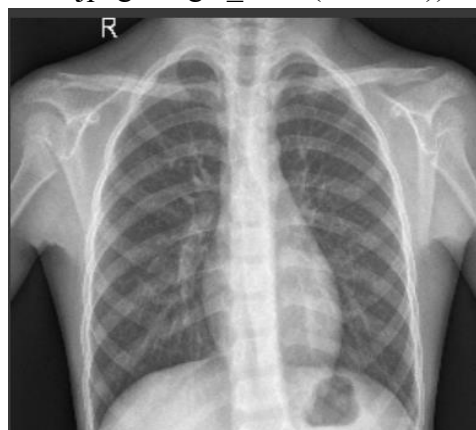
Epoch 1/10
164/164 ----- 56s 281ms/step - accuracy: 0.7987 - loss: 0.6039 - val_accuracy: 0.7821 - val_loss: 0.6733
Epoch 2/10
164/164 ----- 40s 241ms/step - accuracy: 0.9565 - loss: 0.1086 - val_accuracy: 0.6875 - val_loss: 1.6154
Epoch 3/10
164/164 ----- 41s 241ms/step - accuracy: 0.9721 - loss: 0.0752 - val_accuracy: 0.7228 - val_loss: 1.2366
Epoch 4/10
164/164 ----- 43s 253ms/step - accuracy: 0.9825 - loss: 0.0544 - val_accuracy: 0.7917 - val_loss: 0.7992
Epoch 5/10
164/164 ----- 82s 254ms/step - accuracy: 0.9814 - loss: 0.0538 - val_accuracy: 0.7356 - val_loss: 1.7756
Epoch 6/10
164/164 ----- 80s 239ms/step - accuracy: 0.9869 - loss: 0.0332 - val_accuracy: 0.7324 - val_loss: 1.8933
Epoch 7/10
164/164 ----- 41s 241ms/step - accuracy: 0.9934 - loss: 0.0179 - val_accuracy: 0.7019 - val_loss: 1.8058
Epoch 8/10
164/164 ----- 51s 304ms/step - accuracy: 0.9914 - loss: 0.0228 - val_accuracy: 0.7516 - val_loss: 1.7342
Epoch 9/10
164/164 ----- 49s 296ms/step - accuracy: 0.9904 - loss: 0.0266 - val_accuracy: 0.7115 - val_loss: 2.5102
Epoch 10/10
164/164 ----- 73s 243ms/step - accuracy: 0.9965 - loss: 0.0078 - val_accuracy: 0.7131 - val_loss: 2.9657
<keras.src.callbacks.history.History at 0x7f732dcc4a90>

```

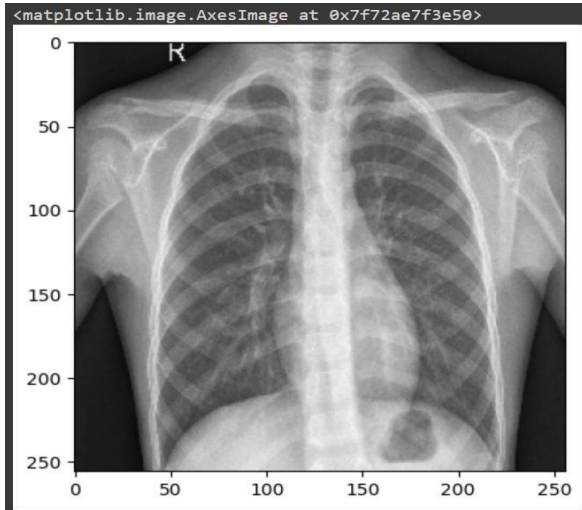
```

import cv2
test_img = keras.utils.load_img('/content/Pediatric Chest X-ray Pneumonia/test/NORMAL/IM-0001-0001.jpeg', target_size=(256,256)) test_img

```



```
from matplotlib import pyplot as plt plt.imshow(test_img)
```



```
import numpy as np test_img  
= np.array(test_img)  
test_img.shape
```

```
(256, 256, 3)
```

```
test_img = test_img.reshape((1,256,256,3)) if  
model.predict(test_img) == 1:  
    print('Pneumonia')  
else:  
    print('Normal')
```

```
1/1 ————— 1s 924ms/step  
Normal
```

Signature: _____

PRACTICAL NO.: 08

AIM: a) Vanilla Auto Encoders

```
import keras
from keras import layers, Input, Model
from keras.layers import Dense
from tensorflow.keras.datasets import mnist
import numpy as np
from tensorflow.keras.utils import plot_model

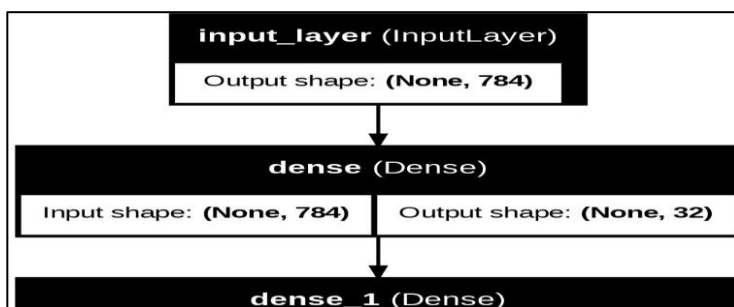
encoding_dim = 32 #Latent Representation
input_img = keras.Input(shape=(784,)) #input image
encoded = layers.Dense(encoding_dim, activation='relu')(input_img) #Encoder representation of input
decoded = layers.Dense(784, activation='sigmoid')(encoded) #Decoder representation of lossy representation of input #output is same as input dim

from json import encoder #creating autoencoder model
autoencoder = Model(input_img, decoded)
#create the encoder model
encoder = Model(input_img, encoded)
#create the decoder model
encoded_input = Input(shape=(encoding_dim,))
decoder_layer = autoencoder.layers[-1]
decoder = Model(encoded_input, decoder_layer(encoded_input))
autoencoder.compile(optimizer='adam', loss='binary_crossentropy')
autoencoder.summary()
plot_model(autoencoder, to_file='autoencoder.png', show_shapes=True, show_layer_names=True)
```

Model: "functional"

Layer (type)	Output Shape	Param #
input_layer (InputLayer)	(None, 784)	0
dense (Dense)	(None, 32)	25,120
dense_1 (Dense)	(None, 784)	25,872

Total params: 50,992 (199.19 KB)
Trainable params: 50,992 (199.19 KB)
Non-trainable params: 0 (0.00 B)



! pip install graphviz

```
#scale and make train and test dataset
(X_train, _), (X_test, _) = mnist.load_data()
X_train = X_train.astype('float32') / 255.0
X_test = X_test.astype('float32') / 255.0
X_train = X_train.reshape((len(X_train), np.prod(X_train.shape[1:])))
X_test = X_test.reshape((len(X_test), np.prod(X_test.shape[1:])))
print(X_train.shape) print(X_test.shape)
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ————— 0s 0us/step
(60000, 784)
(10000, 784)
```

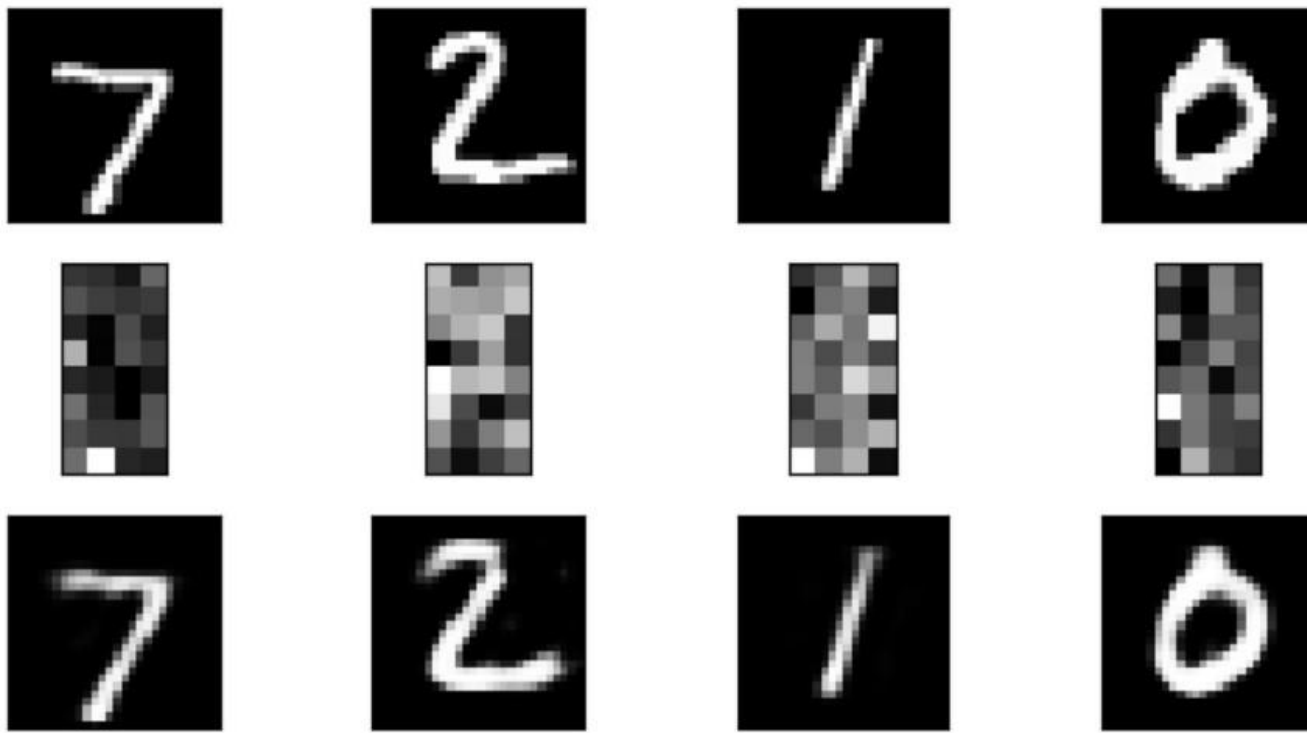
```
autoencoder.fit(X_train, X_train, epochs=50, batch_size=256, shuffle=True,
validation_data=(X_test,
X_test)) #x_train 2 times because it is AutoEncode is USLearning encoded_imgs
= encoder.predict(X_test)
decoded_imgs = decoder.predict(encoded_imgs)
```

```
Epoch 1/50
235/235 ————— 4s 8ms/step - loss: 0.3849 - val_loss: 0.1861
Epoch 2/50
235/235 ————— 2s 2ms/step - loss: 0.1768 - val_loss: 0.1525
Epoch 3/50
235/235 ————— 1s 2ms/step - loss: 0.1486 - val_loss: 0.1338
Epoch 4/50
235/235 ————— 1s 3ms/step - loss: 0.1316 - val_loss: 0.1215
Epoch 5/50
235/235 ————— 1s 3ms/step - loss: 0.1206 - val_loss: 0.1134
Epoch 6/50
235/235 ————— 1s 3ms/step - loss: 0.1132 - val_loss: 0.1074
Epoch 7/50
235/235 ————— 1s 3ms/step - loss: 0.1076 - val_loss: 0.1031
Epoch 8/50
235/235 ————— 1s 2ms/step - loss: 0.1036 - val_loss: 0.1000
Epoch 9/50
235/235 ————— 1s 2ms/step - loss: 0.1003 - val_loss: 0.0976
Epoch 10/50
235/235 ————— 1s 3ms/step - loss: 0.0985 - val_loss: 0.0961
Epoch 11/50
235/235 ————— 1s 2ms/step - loss: 0.0969 - val_loss: 0.0950
Epoch 12/50
235/235 ————— 1s 2ms/step - loss: 0.0958 - val_loss: 0.0943
Epoch 13/50
235/235 ————— 1s 2ms/step - loss: 0.0952 - val_loss: 0.0937
```

```
import matplotlib.pyplot as plt
n = int(input("Enter the number of images to be displayed: ")) #How many digits we will display
plt.figure(figsize=(40, 4)) for i in range(n): # display original ax = plt.subplot(3, 20, i + 1)
plt.imshow(X_test[i].reshape(28, 28))
plt.gray()
ax.get_xaxis().set_visible(False)
ax.get_yaxis().set_visible(False)
#display encoded image ax =
plt.subplot(3, 20, i + 1 + 20)
plt.imshow(encoded_imgs[i].reshape(8, 4))
plt.gray() ax.get_xaxis().set_visible(False)
ax.get_yaxis().set_visible(False) # display
reconstructed ax = plt.subplot(3, 20, 2*20
+ i + 1)
```

```
plt.imshow(decoded_imgs[i].reshape(28, 28))
plt.gray() ax.get_xaxis().set_visible(False)
ax.get_yaxis().set_visible(False)
plt.show()
```

Enter the number of images to be displayed: 4



Signature: _____

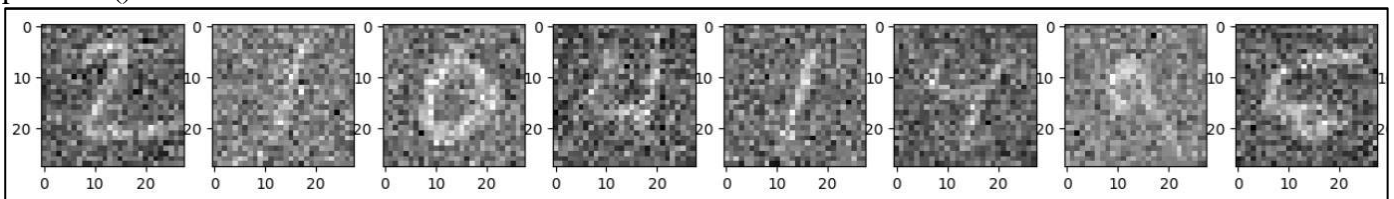
PRACTICAL NO.: 08

AIM: b) Denoising Auto Encoders

```
import keras import numpy as np import
matplotlib.pyplot as plt from
keras.callbacks import TensorBoard from
keras.datasets import mnist from keras
import layers
(X_train, _), (X_test, _) = mnist.load_data() X_train
= X_train.astype('float32') / 255.
X_test = X_test.astype('float32') / 255.
X_train = np.reshape(X_train,(len(X_train),28,28,1)) X_test
= np.reshape(X_test,(len(X_test),28,28,1)) noise_factor =
0.5
X_train_noisy = X_train + noise_factor * np.random.normal(loc=0.0, scale=1.0, size=X_train.shape)
X_test_noisy = X_test + noise_factor * np.random.normal(loc=0.0, scale=1.0, size=X_test.shape)
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ————— 1s 0us/step
```

```
X_train_noisy = np.clip(X_train_noisy, 0., 1.)
X_test_noisy = np.clip(X_test_noisy, 0., 1.) n
= 10 plt.figure(figsize=(20, 5)) for i in
range(1,n+1):
    ax = plt.subplot(1, n, i)
    plt.imshow(X_test_noisy[i].reshape(28, 28))
plt.gray()
    ax.get_xaxis().set_visible(True)
    ax.get_yaxis().set_visible(True)
plt.show()
```



```
from enum import auto
input_img = keras.Input(shape=(28, 28, 1))
x = layers.Conv2D(32, (3, 3), activation='relu', padding='same')(input_img) x
= layers.MaxPooling2D((2, 2), padding='same')(x)
x = layers.Conv2D(32, (3, 3), activation='relu', padding='same')(x) encoded
= layers.MaxPooling2D((2, 2), padding='same')(x)
x = layers.Conv2D(32, (3, 3), activation='relu', padding='same')(encoded) x
= layers.UpSampling2D((2, 2))(x)
x = layers.Conv2D(32, (3,3), activation='relu', padding='same')(x) x
= layers.UpSampling2D((2, 2))(x)
```

```

decoded = layers.Conv2D(1, (3, 3), activation='sigmoid', padding='same')(x)
autoencoder = keras.Model(input_img, decoded)
autoencoder.compile(optimizer='adam', loss='binary_crossentropy')
autoencoder.fit(X_train_noisy,X_train,epochs=3, batch_size=128,
shuffle = True, validation_data=(X_test_noisy,X_test),
callbacks=[TensorBoard(log_dir='/tmo/tb', histogram_freq=0,
write_graph=False)])
predictions = autoencoder.predict(X_test_noisy)

```

```

Epoch 1/3
469/469 ————— 7s 10ms/step - loss: 0.2459 - val_loss: 0.1101
Epoch 2/3
469/469 ————— 7s 6ms/step - loss: 0.1090 - val_loss: 0.1033
Epoch 3/3
469/469 ————— 5s 6ms/step - loss: 0.1033 - val_loss: 0.1007
313/313 ————— 1s 2ms/step

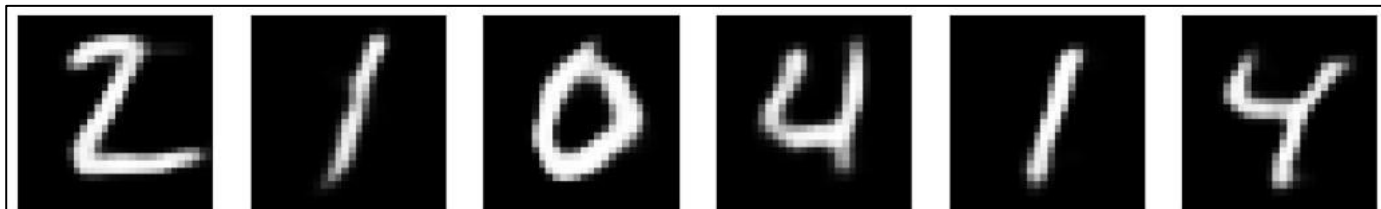
```

```

m=10
plt.figure(figsize=(20, 2))
for i in range(1,m+1):
    ax = plt.subplot(1, m, i)

    plt.imshow(predictions[i].reshape(28, 28))
    plt.gray()
    ax.get_xaxis().set_visible(False)
    ax.get_yaxis().set_visible(False)
plt.show()

```



Signature: _____

PRACTICAL NO.: 09

Date: 03/09/2024 AIM: Generative Adversarial Network (GANs)

```
import torch import
torch.nn as nn import
torch.optim as optim import
torchvision
from torchvision import datasets ,transforms import
matplotlib.pyplot as plt
import numpy as np
#set device
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu') # for checking
#Define a basic transform transform
= transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,0.5,0.5),(0.5,0.5,0.5)) ])
train_dataset = datasets.CIFAR10(root='./data',train=True,download=True,transform=transform)
dataloader = torch.utils.data.DataLoader(train_dataset,batch_size=64,shuffle=True)
```

```
Downloading https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz to ./data/cifar-10-python.tar.gz
100%|██████████| 170498071/170498071 [00:01<00:00, 105986113.74it/s]
Extracting ./data/cifar-10-python.tar.gz to ./data
```

#Hyperparameters

```
latent_dim = 100 lr
= 0.0002
beta1 = 0.5 num_epochs
= 100
beta2 = 0.999
```

#Define the gnerator class

```
Generator(nn.Module): def
__init__(self,latent_dim):
    super(Generator,self).__init__()
    self.model = nn.Sequential(
nn.Linear(latent_dim, 128*8*8),
    nn.ReLU(),
nn.Unflatten(1,(128,8,8)),
nn.Upsample(scale_factor=2),
    nn.Conv2d(128,128,kernel_size=3,stride=1,padding=1),
nn.BatchNorm2d(128, momentum=0.78), nn.ReLU(),
nn.Upsample(scale_factor=2),
    nn.Conv2d(128,64,kernel_size=3,padding=1),
    nn.BatchNorm2d(64, momentum=0.78),
nn.ReLU(),
nn.Conv2d(64,3,kernel_size=3,padding=1),
nn.Tanh() )
def forward(self,z):
img = self.model(z)
return img
```

#Define discriminator

```
class Discriminator(nn.Module):    def
__init__(self):
super(Discriminator,self).__init__()
    self.model = nn.Sequential(
        nn.Conv2d(3,32,kernel_size=3,stride=2,padding=1),
nn.LeakyReLU(0.2),        nn.Dropout(0.25),
        nn.Conv2d(32,64,kernel_size=3,stride=2,padding=1),
        nn.ZeroPad2d((0,1,0,1)),
nn.BatchNorm2d(64, momentum=0.82),
        nn.LeakyReLU(0.25),
nn.Dropout(0.25),
        nn.Conv2d(64,128,kernel_size=3,stride=2,padding=1),
nn.BatchNorm2d(128, momentum=0.82),
        nn.LeakyReLU(0.25),
nn.Dropout(0.25),
        nn.Conv2d(128,256,kernel_size=3,stride=1,padding=1),
nn.BatchNorm2d(256, momentum=0.82),
        nn.LeakyReLU(0.25),
nn.Dropout(0.25),        nn.Flatten(),
nn.Linear(256*5*5,1),
nn.Sigmoid()    )
    def forward(self,img):
        validity = self.model(img)
        return validity
```

#Define the generator and discriminator

```
#Initialize generator and discriminator generator
= Generator(latent_dim).to(device) discriminator =
Discriminator().to(device)
```

#Loss fuction

```
adversarial_loss = nn.BCELoss()
```

#Optmizers

```
optimizers_G = optim.Adam(generator.parameters(),lr=lr,betas=(beta1,beta2)) optimizers_D
= optim.Adam(discriminator.parameters(),lr=lr,betas=(beta1,beta2))
```

#Training loop for epoch in

```
range(num_epochs):    for i, batch in
enumerate(dataloader):    #Convert
```

list to tensor

```
    real_images = batch[0].to(device)    #Adversarial
ground truths    valid = torch.ones(real_images.size(0),1,
device=device)    fake = torch.zeros(real_images.size(0),1,
device=device)
```

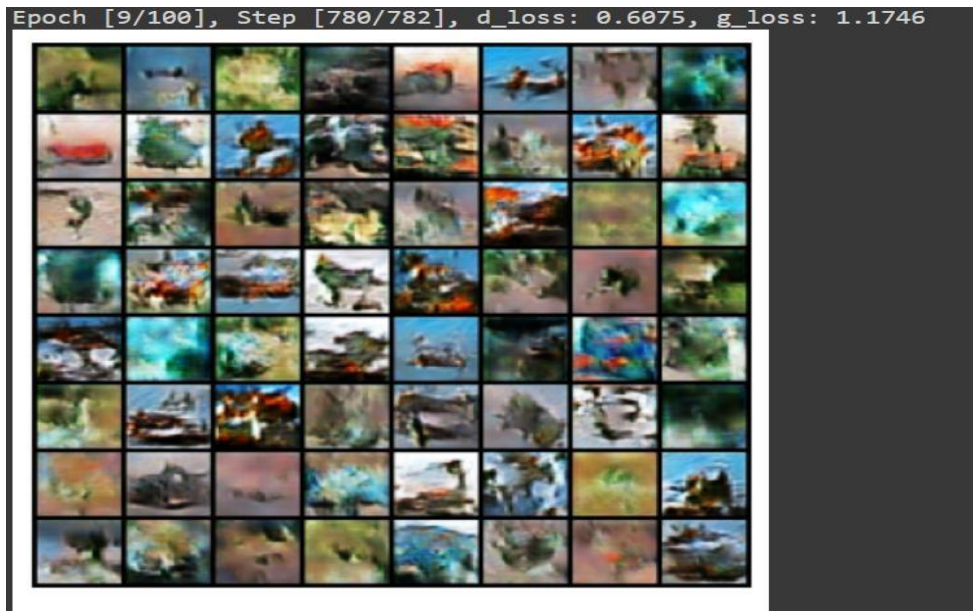
#Configure input

```
real_images = real_images
optimizers_D.zero_grad()    #Sample noise as generator input
    z = torch.randn(real_images.size(0),latent_dim,device=device)
    #Generate a batch of images
fake_images = generator(z)
```

```

#Measure discriminator's ability
real_loss = adversarial_loss(discriminator(real_images),valid)
fake_loss = adversarial_loss(discriminator(fake_images.detach()),fake)
d_loss = (real_loss + fake_loss)/2
#Backward pass and optimize
d_loss.backward()
optimizers_D.step()
#Train the generator
optimizers_G.zero_grad() #Generate
a batch of images
gen_images = generator(z)
#Adversarial loss
g_loss = adversarial_loss(discriminator(gen_images),valid)
#Backward pass and optimize
g_loss.backward()
optimizers_G.step()
#Progress Mointoring
if (i+1)%10 == 0: print(f'Epoch [{epoch+1}/{num_epochs}], Step [{i+1}/{len(dataloader)}], d_loss:
{d_loss.item():.4f}, g_loss: {g_loss.item():.4f}')
#Save generated images for every epoch if
(epoch+1)%10 == 0: with torch.no_grad():
z = torch.randn(64,latent_dim,device=device)
generated = generator(z).detach().cpu()
grid = torchvision.utils.make_grid(generated,nrow=8,normalize=True)
plt.imshow(np.transpose(grid,(1,2,0))) plt.axis('off') plt.show()

```



Signature: _____